

Performance-Tuning-Parameter für Transformationen

Section 3 · Data Transformation and Modelling · Quelle: PySpark-/Databricks-Referenzdokumentation

Neben der korrekten Formulierung von Joins, Filtern und Aggregationen (Kapitel 11–13 dieser Section) verlangt der Exam Guide auch ein grundlegendes Verständnis der wichtigsten Spark-Tuning-Parameter: `spark.sql.shuffle.partitions`, `spark.default.parallelism`, die Speicherparameter `spark.executor.memory` / `spark.driver.memory` sowie `spark.sql.autoBroadcastJoinThreshold`. Dieses Kapitel erklärt jeden Parameter, nennt typische Werte und zeigt, wie sich eine Konfigurationsänderung anschließend „vermessen“ lässt.

1. `spark.sql.shuffle.partitions`

Dieser Parameter legt fest, in wie viele Partitionen Daten bei einem Shuffle (also bei Wide Transformations wie `join()`, `groupBy()` oder `distinct()`) aufgeteilt werden. Der Standardwert beträgt **200** – ein Wert, der historisch für mittelgroße On-Premises-Cluster gewählt wurde und für viele moderne Workloads nicht mehr optimal ist. Zu viele Shuffle-Partitionen bei kleinen Datenmengen erzeugen unnötigen Overhead durch viele kleine Tasks; zu wenige Partitionen bei großen Datenmengen führen zu übergroßen Partitionen und damit potenziell zu Spill oder Out-of-Memory-Fehlern.

```
# aktuellen Wert auslesen
print(spark.conf.get("spark.sql.shuffle.partitions")) # Standard: 200

# Wert anpassen, z.B. für einen kleineren Cluster mit wenigen Cores
spark.conf.set("spark.sql.shuffle.partitions", 32)
```

```
SET spark.sql.shuffle.partitions = 32;
```

Auf Databricks übernimmt **Adaptive Query Execution (AQE)** seit Spark 3.0 einen Großteil dieser Feinabstimmung automatisch: `spark.sql.shuffle.partitions` wird dabei zur Laufzeit dynamisch angepasst (Coalescing kleiner Partitionen), sodass eine manuelle Anpassung in den meisten Fällen

gar nicht mehr notwendig ist. Für Workloads ohne AQE bzw. zum grundlegenden Verständnis bleibt der Parameter dennoch relevant.

2. spark.default.parallelism

Während `spark.sql.shuffle.partitions` ausschließlich für DataFrame-/SQL-Shuffles gilt, steuert `spark.default.parallelism` die Standard-Parallelität von **RDD**-Operationen (etwa `parallelize()` oder RDD-Joins/-Aggregationen) sowie den Default für Operationen ohne explizite Partitionsangabe. Der Standardwert richtet sich nach der Cluster-Größe: bei lokalem Modus nach der Anzahl der Cores der Maschine, im Cluster-Modus nach der Gesamtzahl aller verfügbaren Executor-Cores.

```
print(spark.sparkContext.defaultParallelism) # z.B. 8 auf einer 8-Core-Maschine
spark.conf.set("spark.default.parallelism", 100)
```

In der täglichen Arbeit mit DataFrames (statt RDDs) spielt `spark.default.parallelism` eine untergeordnete Rolle gegenüber `spark.sql.shuffle.partitions` – für das Verständnis der Beziehung zwischen Cluster-Größe und Standard-Parallelität ist der Parameter dennoch Teil des Grundwissens.

3. Speicherparameter: spark.executor.memory und spark.driver.memory

`spark.executor.memory` legt fest, wie viel Arbeitsspeicher jedem einzelnen Executor-Prozess für die Ausführung von Tasks zur Verfügung steht; `spark.driver.memory` entsprechend für den Treiberprozess, der u. a. den Ausführungsplan koordiniert und Ergebnisse von `collect()`-Aufrufen empfängt. Auf Databricks werden diese Werte in der Regel automatisch anhand des gewählten Cluster-/Instanztyps gesetzt, lassen sich bei Bedarf aber über die Cluster-Konfiguration (Spark-Config) oder programmatisch überschreiben.

```
# Hinweis: Speicherparameter müssen i.d.R. beim Start der Spark-Session/des Clusters
# gesetzt werden, da Executors bereits mit einer festen Speichergrenze starten.
spark.conf.set("spark.executor.memory", "8g")
spark.conf.set("spark.driver.memory", "4g")
```

Ein zu knapp bemessenes `spark.executor.memory` äußert sich typischerweise in vermehrtem Spill (Daten werden auf Festplatte ausgelagert) oder in Out-of-Memory-Fehlern einzelner Tasks; ein zu

knapp bemessenes `spark.driver.memory` macht sich vor allem bei großen `collect()`-Aufrufen oder komplexen Ausführungsplänen mit vielen Partitionen bemerkbar.

4. spark.sql.autoBroadcastJoinThreshold

Dieser Parameter bestimmt, ab welcher geschätzten Tabellengröße Spark bei einem Join *automatisch* einen Broadcast Join wählt (siehe Kapitel 11 zu Join-Typen), statt einen teuren Shuffle beider Tabellen durchzuführen. Der Standardwert beträgt **10 MB (10485760 Bytes)**. Wird der Wert auf `-1` gesetzt, deaktiviert dies den automatischen Broadcast vollständig.

```
print(spark.conf.get("spark.sql.autoBroadcastJoinThreshold")) # Standard: 10485760 (10 MB)

# Schwellenwert erhöhen, z.B. auf 100 MB, wenn genügend Executor-Speicher vorhanden ist
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 100 * 1024 * 1024)

# automatischen Broadcast vollständig deaktivieren
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

```
SET spark.sql.autoBroadcastJoinThreshold = 104857600; -- 100 MB
```

Ein höherer Schwellenwert kann bei Joins mit mittelgroßen Dimensionstabellen sinnvoll sein, erhöht aber auch das Risiko von Speicherproblemen auf den Executoren, falls die geschätzte Tabellengröße die tatsächliche unterschätzt (etwa nach komplexen vorgelagerten Transformationen).

5. Eine Änderung „re-messen“: Vorgehen in der Praxis

Der Exam Guide verlangt ausdrücklich, eine Parameteränderung anschließend hinsichtlich ihrer Wirkung zu überprüfen. In einem Databricks-Notebook bieten sich dafür zwei ergänzende Methoden an:

1. **Zeitmessung mit `%timeit`**: In einer Python-Notebookzelle lässt sich ein Transformationsschritt (inkl. einer abschließenden Aktion wie `count()`, da Spark sonst lazy bleibt) mehrfach automatisiert ausführen und die durchschnittliche Laufzeit ausgeben.
2. **Vergleich in der Spark-UI**: Über den Reiter „SQL / DataFrame“ lässt sich für jede Ausführung die Gesamtlaufzeit sowie – über die Stage-Detailseiten – Kennzahlen wie Shuffle Read/Write-Volumen und Spill einsehen und zwischen „vorher“ und „nachher“ vergleichen.

```
# Beispiel: Auswirkung einer geänderten Shuffle-Partitionsanzahl messen
spark.conf.set("spark.sql.shuffle.partitions", 200)
%timeit grosse_tabelle.groupBy("kategorie").count().collect()

spark.conf.set("spark.sql.shuffle.partitions", 16)
%timeit grosse_tabelle.groupBy("kategorie").count().collect()

# %timeit fuehrt die Zelle standardmaessig mehrfach aus und meldet Mittelwert
# und Standardabweichung der Laufzeit -- so laesst sich der Effekt der
# Konfigurationsaenderung direkt quantitativ vergleichen.
```

Wichtig für belastbare Messungen: Zwischen den beiden Messungen sollte möglichst nur der zu testende Parameter verändert werden, und Caching-Effekte (etwa bereits im Speicher gehaltene Zwischenergebnisse) sollten berücksichtigt bzw. durch mehrfache Wiederholung ausgeglichen werden. Bei kleinen Testdatensmengen – wie sie in Übungsumgebungen üblich sind – fallen Unterschiede durch Tuning-Parameter oft nur gering aus; die eigentliche Wirkung zeigt sich meist erst bei produktionsnahen Datensmengen.

Ergänzt aus der offiziellen Spark-Dokumentation: Der „Performance Tuning“-Leitfaden von Apache Spark bestätigt für `spark.sql.shuffle.partitions` den Standardwert von 200 sowie den Hinweis, dass dieser Wert bei aktivierter Adaptive Query Execution (`spark.sql.adaptive.enabled`) zur Laufzeit automatisch angepasst wird. Für `spark.sql.autoBroadcastJoinThreshold` nennt die Dokumentation den Standardwert 10 MB und weist ausdrücklich darauf hin, dass der Wert `-1` den automatischen Broadcast vollständig deaktiviert; aktuell werden zudem nur Tabellen unterstützt, für die `ANALYZE TABLE ... COMPUTE STATISTICS` ausgeführt wurde oder deren Größe der Dateistatistik entnommen werden kann.

Quelle: [Performance Tuning – Apache Spark Dokumentation](#)